

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет «МИЭТ»

Институт Системной и программной инженерии
и информационных технологий

Дисциплина: Основы программной инженерии

Отчет по лабораторной работе №3

«Интеграционное тестирование»

Вариант 3

Выполнил:

Студент группы П-42

Варлапов Артем

Москва, 2026

Цель работы

Научиться выполнять интеграционное тестирование программного обеспечения на языке Java с использованием JUnit 4.

Задание

Сформировать иерархию классов вокруг ранее протестированного класса. Провести модульное тестирование написанных классов и инкрементальное интеграционное тестирование сверху вниз, обязательно используя программные заглушки для имитации ещё не подключённых классов нижнего уровня.

Для варианта 3 исходным является класс «Строка» (TextLine) из лабораторной работы №2. В лабораторной работе №3 отдельного списка вариантов нет. Упоминание в задании класса из лабораторной №3 трактуется как ссылка на ранее протестированный класс из лабораторной №2.

Выполнение работы

1. Иерархия и взаимодействие классов

TextLine — базовый класс строки. RussianTextLine и EnglishTextLine наследуют TextLine и задают язык при создании. TextDocument — агрегирующий класс: хранит список объектов TextLine и суммирует их текущие длины. Методы базового класса, включая изменение языка, доступны наследникам без дополнительных ограничений.

Связи классов: RussianTextLine → TextLine; EnglishTextLine → TextLine — наследование. TextDocument → список TextLine — агрегация. При вычислении длины документа вызывается getLength() каждого объекта строки.

2. Реализация классов

Базовый класс TextLine перенесён из лабораторной №2 без изменений. Некорректные аргументы обрабатываются через IllegalArgumentException.

TextLine.java

```
import java.util.Arrays;

public class TextLine {
    private int length;
    private char[] characters;
    private String language;

    public TextLine(String text, String language) {
        if (text == null) {
            throw new IllegalArgumentException("Text must not be null");
        }
        setLanguage(language);
        characters = text.toCharArray();
        length = characters.length;
    }

    public int getLength() {
        return length;
    }

    public void setLength(int length) {
        if (length < 0) {
            throw new IllegalArgumentException("Length must not be negative");
        }
        characters = Arrays.copyOf(characters, length);
        this.length = characters.length;
    }

    public String getLanguage() {
        return language;
    }
}
```

```

public void setLanguage(String language) {
    if (language == null || language.trim().isEmpty()) {
        throw new IllegalArgumentException("Language must not be empty");
    }
    this.language = language;
}

public void append(TextLine other) {
    if (other == null) {
        throw new IllegalArgumentException("Other line must not be null");
    }
    char[] result = Arrays.copyOf(characters, length + other.length);
    System.arraycopy(other.characters, 0, result, length, other.length);
    characters = result;
    length = characters.length;
}
}

```

RussianTextLine.java

```

public class RussianTextLine extends TextLine {
    public RussianTextLine(String text) {
        super(text, "Русский");
    }
}

```

EnglishTextLine.java

```

public class EnglishTextLine extends TextLine {
    public EnglishTextLine(String text) {
        super(text, "Английский");
    }
}

```

TextDocument.java

```

import java.util.ArrayList;
import java.util.List;

public class TextDocument {
    private final List<TextLine> lines = new ArrayList<>();

    public void addLine(TextLine line) {
        if (line == null) {
            throw new IllegalArgumentException("Line must not be null");
        }
        lines.add(line);
    }

    public int getLength() {
        int length = 0;
        for (TextLine line : lines) {
            length += line.getLength();
        }
        return length;
    }
}

```

3. Заглушка и тестирование сверху вниз

На первом этапе проверяется верхний класс `TextDocument`. Настоящие строки заменяются объектами `TextLineStub`. Переопределённый `getLength()` возвращает заданное тестом число независимо от внутреннего массива символов. Наследование используется для совместимости с интерфейсом `TextLine`; конструктор базового класса создаёт пустую строку.

TextLineStub.java

```

// Программная заглушка: заменяет настоящую строку в тестах документа.
public class TextLineStub extends TextLine {
    private final int fixedLength;
}

```

```

public TextLineStub(int fixedLength) {
    super("", "Заглушка");
    this.fixedLength = fixedLength;
}

@Override
public int getLength() {
    return fixedLength;
}
}

```

Этап 1. `TextDocumentTest` проверяет пустой документ, одну заглушку, сумму длин нескольких заглушек и отклонение `null`. Так проверяется логика документа до подключения настоящих строк.

Этап 2. В тесте `documentWorksWithRealTextLines` заглушки заменяются настоящими объектами `TextLine`. Проверяется сумма длин строк «abc» и «de»: 5 символов.

Этап 3. В тестах `documentWorksWithBothSubclasses` и `documentReflectsChangesInItsLines` подключаются наследники. Проверяется сумма длин русской и английской строк, а также изменение длины документа после `append()` и `setLength()` у сохранённой строки.

Каждый тест создаёт собственные объекты и не зависит от порядка выполнения других тестов. Этапы означают последовательное расширение состава проверяемой системы, а не зависимость тестов друг от друга.

4. Модульные тесты

Написаны тесты всех четырёх классов программы: `TextLine` — 14 тестов, `RussianTextLine` — 3, `EnglishTextLine` — 3, `TextDocument` — 4. Всего 24 модульных теста. Проверяются корректные данные, пустые строки, граничные длины и ошибочные аргументы.

TextLineTest.java

```

import java.lang.reflect.Field;
import org.junit.Test;

import static org.junit.Assert.*;

public class TextLineTest {
    // Массив закрыт по условию; читаем его только внутри тестов.
    private char[] charactersOf(TextLine line) throws Exception {
        Field field = TextLine.class.getDeclaredField("characters");
        field.setAccessible(true);
        return (char[]) field.get(line);
    }

    @Test
    public void constructorStoresFields() throws Exception {
        TextLine line = new TextLine("Привет", "Русский");
        assertEquals(6, line.getLength());
        assertEquals("Русский", line.getLanguage());
        assertEquals("Привет".toCharArray(), charactersOf(line));
    }

    @Test
    public void emptyTextHasZeroLength() throws Exception {
        TextLine line = new TextLine("", "Русский");
        assertEquals(0, line.getLength());
        assertEquals(new char[0], charactersOf(line));
    }

    @Test
    public void languageCanBeChanged() {
        TextLine line = new TextLine("abc", "Русский");
        line.setLanguage("Английский");
        assertEquals("Английский", line.getLanguage());
        assertEquals(3, line.getLength());
    }
}

```

```

@Test
public void lengthCanBeReducedIncreasedAndSetToZero() throws Exception {
    TextLine line = new TextLine("abc", "Английский");
    line.setLength(2);
    assertEquals(2, line.getLength());
    assertEquals(new char[] {'a', 'b'}, charactersOf(line));
    line.setLength(4);
    assertEquals(4, line.getLength());
    assertEquals(new char[] {'a', 'b', '\0', '\0'}, charactersOf(line));
    line.setLength(0);
    assertEquals(0, line.getLength());
    assertEquals(new char[0], charactersOf(line));
}

@Test
public void appendCopiesCharactersAndRecalculatesLength() throws Exception {
    TextLine first = new TextLine("Привет, ", "Русский");
    TextLine second = new TextLine("world", "Английский");
    first.append(second);
    assertEquals(13, first.getLength());
    assertEquals("Привет, world".toCharArray(), charactersOf(first));
    assertEquals("Русский", first.getLanguage());
    assertEquals(5, second.getLength());
    assertEquals("world".toCharArray(), charactersOf(second));
}

@Test
public void appendHandlesEmptyLines() throws Exception {
    TextLine line = new TextLine("", "Английский");
    line.append(new TextLine("abc", "Английский"));
    line.append(new TextLine("", "Английский"));
    assertEquals(3, line.getLength());
    assertEquals("abc".toCharArray(), charactersOf(line));
}

@Test
public void lineCanBeAppendedToItself() throws Exception {
    TextLine line = new TextLine("ab", "Английский");
    line.append(line);
    assertEquals(4, line.getLength());
    assertEquals("abab".toCharArray(), charactersOf(line));
}

@Test(expected = IllegalArgumentException.class)
public void constructorRejectsNullText() {
    new TextLine(null, "Русский");
}

@Test(expected = IllegalArgumentException.class)
public void constructorRejectsNullLanguage() {
    new TextLine("abc", null);
}

@Test(expected = IllegalArgumentException.class)
public void constructorRejectsEmptyLanguage() {
    new TextLine("abc", "");
}

@Test(expected = IllegalArgumentException.class)
public void setterRejectsBlankLanguage() {
    new TextLine("abc", "Русский").setLanguage(" ");
}

@Test(expected = IllegalArgumentException.class)
public void setterRejectsNullLanguage() {
    new TextLine("abc", "Русский").setLanguage(null);
}

@Test(expected = IllegalArgumentException.class)
public void lengthCannotBeNegative() {
    new TextLine("abc", "Английский").setLength(-1);
}

```

```

@Test(expected = IllegalArgumentException.class)
public void appendRejectsNull() {
    new TextLine("abc", "Английский").append(null);
}
}

```

RussianTextLineTest.java

```

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class RussianTextLineTest {
    @Test
    public void constructorSetsLanguageAndLength() {
        RussianTextLine line = new RussianTextLine("Привет");
        assertEquals("Русский", line.getLanguage());
        assertEquals(6, line.getLength());
    }

    @Test
    public void emptyLineHasZeroLength() {
        assertEquals(0, new RussianTextLine("").getLength());
    }

    @Test(expected = IllegalArgumentException.class)
    public void constructorRejectsNullText() {
        new RussianTextLine(null);
    }
}

```

EnglishTextLineTest.java

```

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class EnglishTextLineTest {
    @Test
    public void constructorSetsLanguageAndLength() {
        EnglishTextLine line = new EnglishTextLine("hello");
        assertEquals("Английский", line.getLanguage());
        assertEquals(5, line.getLength());
    }

    @Test
    public void emptyLineHasZeroLength() {
        assertEquals(0, new EnglishTextLine("").getLength());
    }

    @Test(expected = IllegalArgumentException.class)
    public void constructorRejectsNullText() {
        new EnglishTextLine(null);
    }
}

```

TextDocumentTest.java

```

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class TextDocumentTest {
    @Test
    public void emptyDocumentHasZeroLength() {
        assertEquals(0, new TextDocument().getLength());
    }

    @Test
    public void documentUsesLengthProvidedByStub() {
        TextDocument document = new TextDocument();
        document.addLine(new TextLineStub(7));
        assertEquals(7, document.getLength());
    }
}

```

```

@Test
public void documentSumsStubLengths() {
    TextDocument document = new TextDocument();
    document.addLine(new TextLineStub(3));
    document.addLine(new TextLineStub(0));
    document.addLine(new TextLineStub(5));
    assertEquals(8, document.getLength());
}

@Test(expected = IllegalArgumentException.class)
public void documentRejectsNullLine() {
    new TextDocument().addLine(null);
}
}

```

5. Интеграционные тесты

Три интеграционных теста проверяют взаимодействие документа с настоящим базовым классом и наследниками, включая пересчёт общей длины после изменения строки.

TextDocumentIntegrationTest.java

```

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class TextDocumentIntegrationTest {
    // Этап 2: заглушки заменены настоящими объектами базового класса.
    @Test
    public void documentWorksWithRealTextLines() {
        TextDocument document = new TextDocument();
        document.addLine(new TextLine("abc", "Английский"));
        document.addLine(new TextLine("de", "Английский"));
        assertEquals(5, document.getLength());
    }

    // Этап 3: подключены классы-наследники.
    @Test
    public void documentWorksWithBothSubclasses() {
        TextDocument document = new TextDocument();
        document.addLine(new RussianTextLine("Привет"));
        document.addLine(new EnglishTextLine("hello"));
        assertEquals(11, document.getLength());
    }

    @Test
    public void documentReflectsChangesInItsLines() {
        TextDocument document = new TextDocument();
        RussianTextLine line = new RussianTextLine("Привет");
        document.addLine(line);
        assertEquals(6, document.getLength());
        line.append(new RussianTextLine("!"));
        assertEquals(7, document.getLength());
        line.setLength(2);
        assertEquals(2, document.getLength());
    }
}

```

6. Результаты тестирования

Сначала отдельно запущена проверка верхнего класса с заглушками. Все 4 теста прошли успешно.

JUnit version 4.13.2

....

Time: 0.008

OK (4 tests)

Рисунок 1 — Проверка TextDocument с программными заглушками

Затем запущен полный набор модульных и интеграционных тестов через JUnitCore из JUnit 4.13.2. Все 27 тестов прошли успешно. Ниже приведён фактический вывод запуска.

JUnit version 4.13.2

.....

Time: 0.015

OK (27 tests)

Рисунок 2 — Результат выполнения полного набора тестов

Вывод

В ходе работы построена иерархия вокруг класса «Строка» и добавлен агрегирующий класс `TextDocument`. Проведено модульное тестирование и инкрементальное интеграционное тестирование сверху вниз: от документа с заглушками к документу с настоящими строками и наследниками. Подтверждена корректность передачи длины между объектами и её изменения после операций со строкой. Все 24 модульных и 3 интеграционных теста пройдены успешно.